

CS485

Data Science and Applications

Lecture 1: Introduction

Announcement

NO lecture this Thursday (13/2/2026)



Today's Objectives

- Course overview
- Introduction to topic
- Python primer
- Useful python libraries



About CS485

- Lectures A113

- Tuesday/Thursday 9:00-11:00

- TA

- Τζήμας Ιάσων, csdp1333@csd.uoc.gr

- Φώτης Ιωάννης, csdp1350@csd.uoc.gr

- Κυριακού Ειρήνη, csdp1359@csd.uoc.gr

- Δρενοβιάδης Κωνσταντίνος, csdp1434@csd.uoc.gr

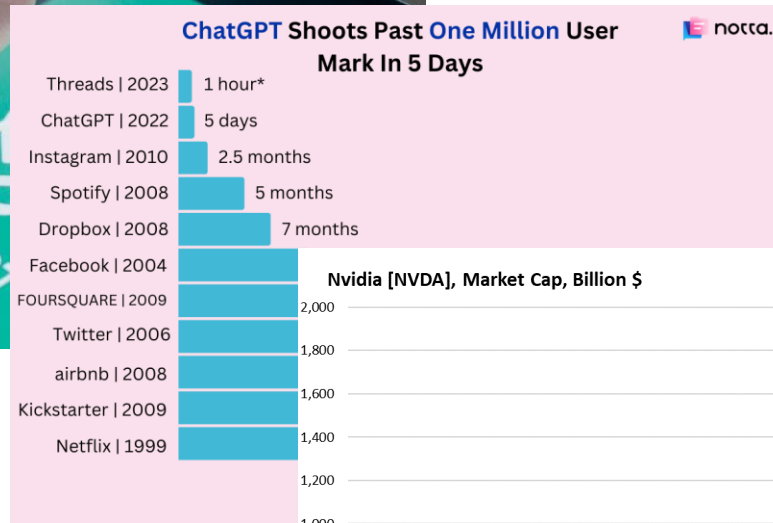
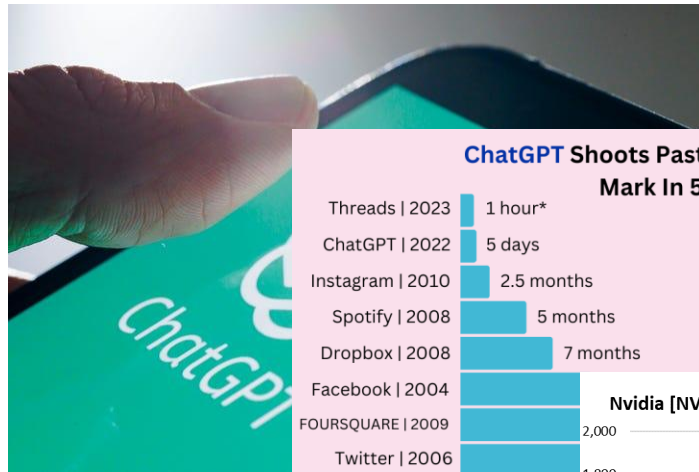
- Prerequisites: HY-119 (Linear Algebra), HY-150 (Programming), HY-217 (Probabilities)

- Course material

- <https://elearn.uoc.gr/course/>



Motivation



Nvidia [NVDA], Market Cap, Billion \$



Source: YCharts



Course Topics

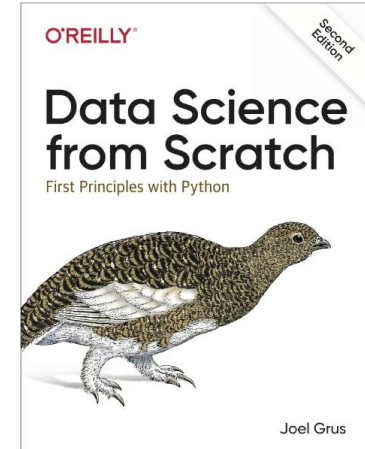
Week		Short description
1	10-Feb	Introduction to Data science & Python primer
2	17-Feb	Encoding data: vector, matrices
3	24-Feb	Optimization: Key Algorithms & Methods
4	3-Mar	Probability & Statistics
5	10-Mar	Learning from Data: Machine Learning Principles
6	17-Mar	Deep Learning Foundations
7	24-Mar	Supervised Learning
8	31-Mar	Unsupervised & Generative Learning
9	21-Apr	Image Analytics (CNN)
10	28-Apr	Time Series Analysis
11	5-May	Natural Language Processing
12	12-May	Tabular data processing
13	19-May	Reinforcement Learning



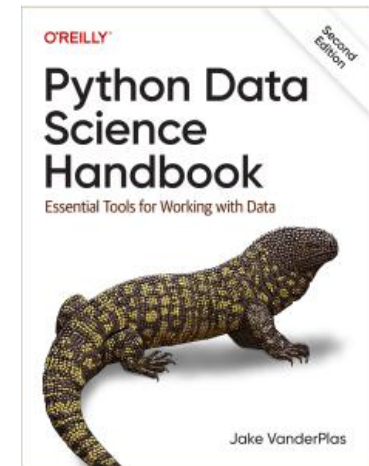
Books

- Grus Joel, Επιστήμη Δεδομένων: Βασικές Αρχές και Εφαρμογές με Python, 2η έκδοση, Εκδόσεις Παπασωτηρίου, 2021. (Μετάφραση του Data Science from Scratch: First Principles with Python 2nd Edition, O'Reilly Media)

Κωδικός Βιβλίου στον Εύδοξο: 94690736



- Jake VanderPlas - Python Data Science Handbook- O'Reilly Media (2022)
- Βερύκιος, Β., Καγκλής, Β., & Σταυρόπουλος, Η. (2015). Η επιστήμη των δεδομένων μέσα από τη γλώσσα R [Προπτυχιακό εγχειρίδιο]. Κάλλιπος, <https://hdl.handle.net/11419/2965> (suggested)
- Jeff M. Phillips, Mathematical Foundations for Data Analysis, Springer Series in the Data Sciences, 1st ed. 2021 Edition <https://mathfordata.github.io/> (suggested)



Grading

- Weekly assignments: 40%
 - Jupiter notebook with comments/text
- Quizzes/Midterm: 30%
 - In class, online, every few weeks
- Final exam: 30%
 - Multiple choice + open-ended

All above are compulsory for getting a grade at the end of the exam



Digital Crete Challenge

- Multimodal dataset
- Multiple tracks
- 3 phases $\leftrightarrow$ aligned with covered material

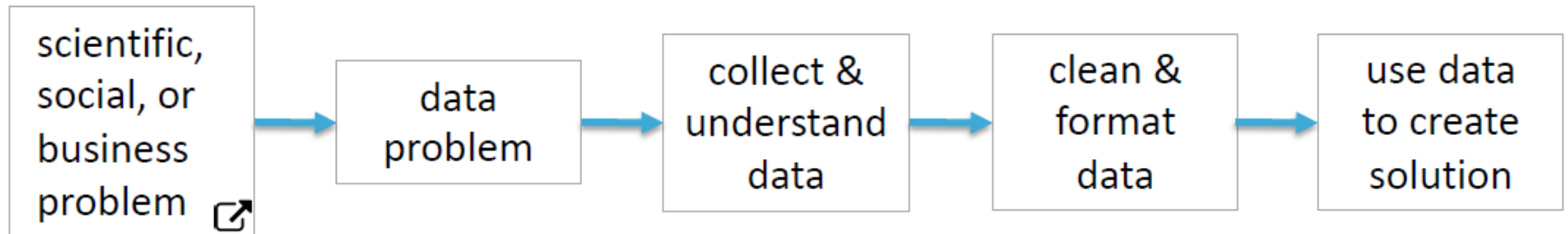
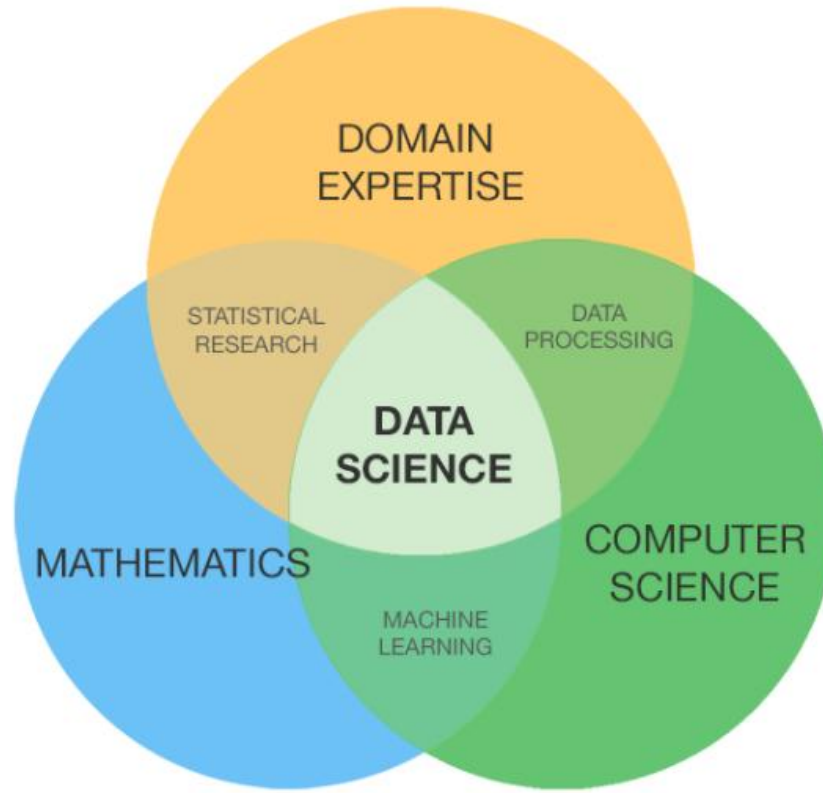


* Gemini

Weekly assignments

Week	Short description
1	Perform data manipulation and basic analysis on a sample dataset using Pandas and create visualizations using Matplotlib and Seaborn.
2	Perform linear and non-linear regression on real data
3	Perform statistical analysis of real-world datasets, including hypothesis testing.
4	Build and validate machine learning models using scikit-learn.
5	Create and train a neural network using TensorFlow.
6	Develop a data cleaning and preprocessing pipeline (handling missing values and outliers).
7	Deploy a machine learning model on a cloud platform and create an API for predictions.
8	Analyze unstructured data formats with Pandas.
9	Apply signal processing techniques for noise reduction in audio.
10	Build a time series forecasting model using techniques like ARIMA.
11	Implement an image classification task using a pre-trained neural network.
12	Sentiment analysis using libraries like NLTK or spaCy.
13	Analyze network data using graph libraries (NetworkX) for identifying key nodes and communities.





What Is Data Analysis?

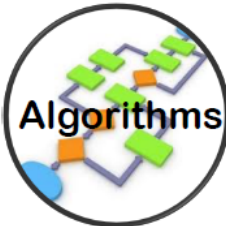
...using data to discover useful information...



- **data:** anything you can *measure* or *record*



- **statistics:** summarize (and visualize) *main characteristics* of the data



- **algorithms:** apply algorithms to find *patterns* in the data

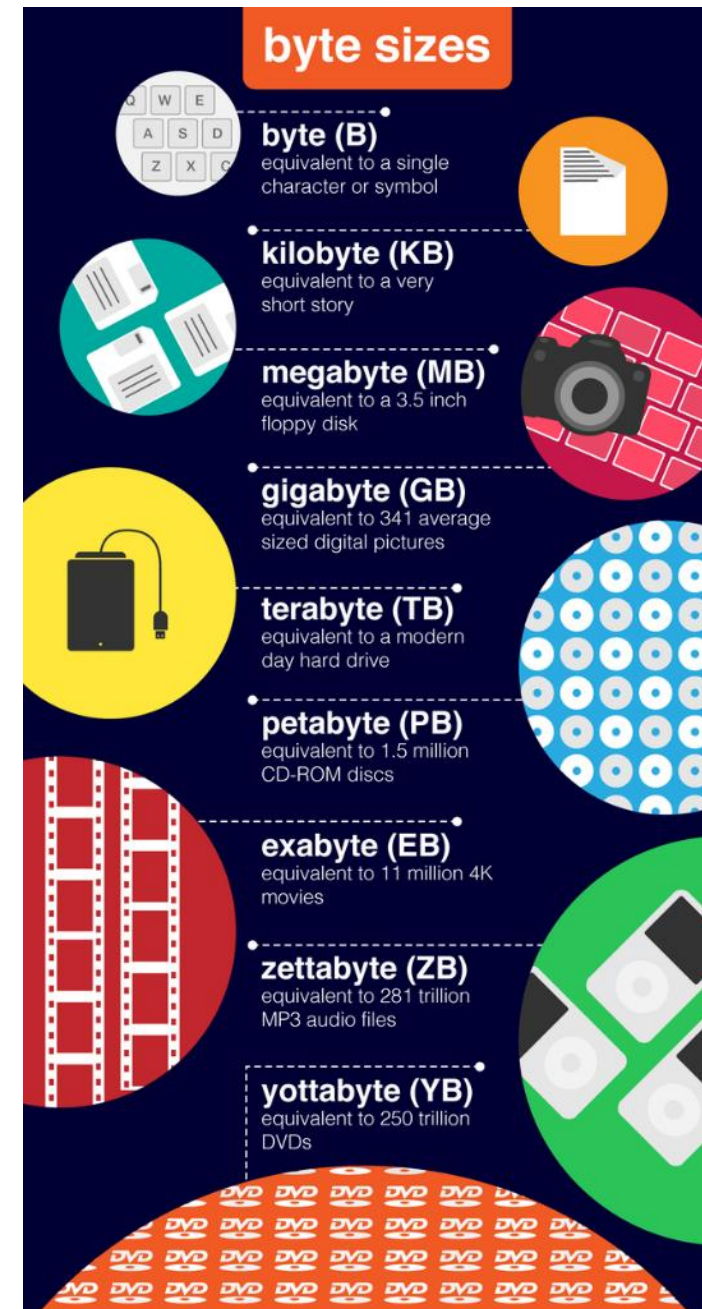
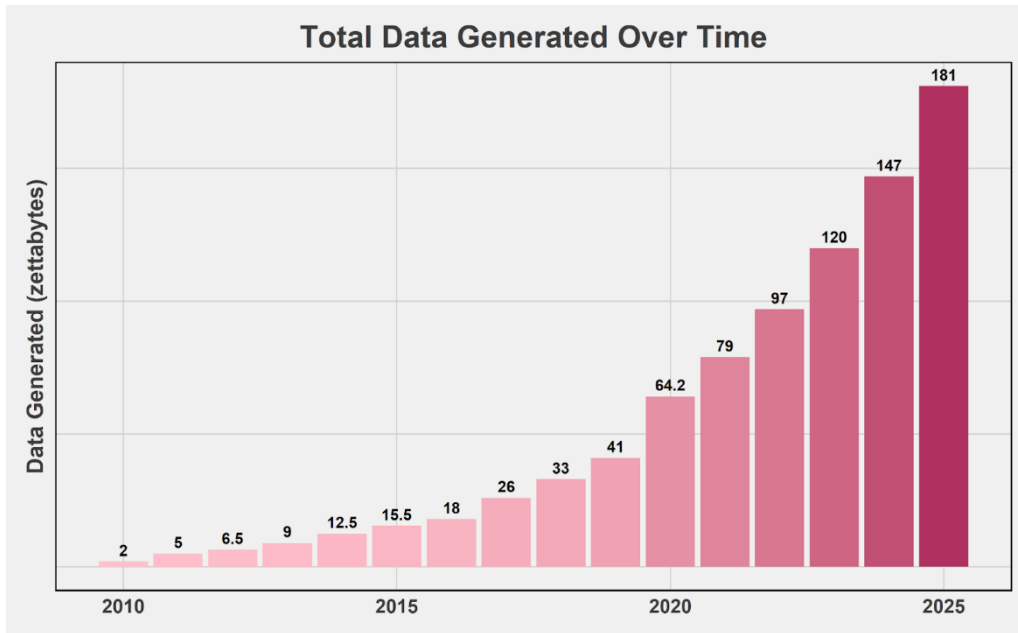


“Big Data”

Big Data

The 5Vs

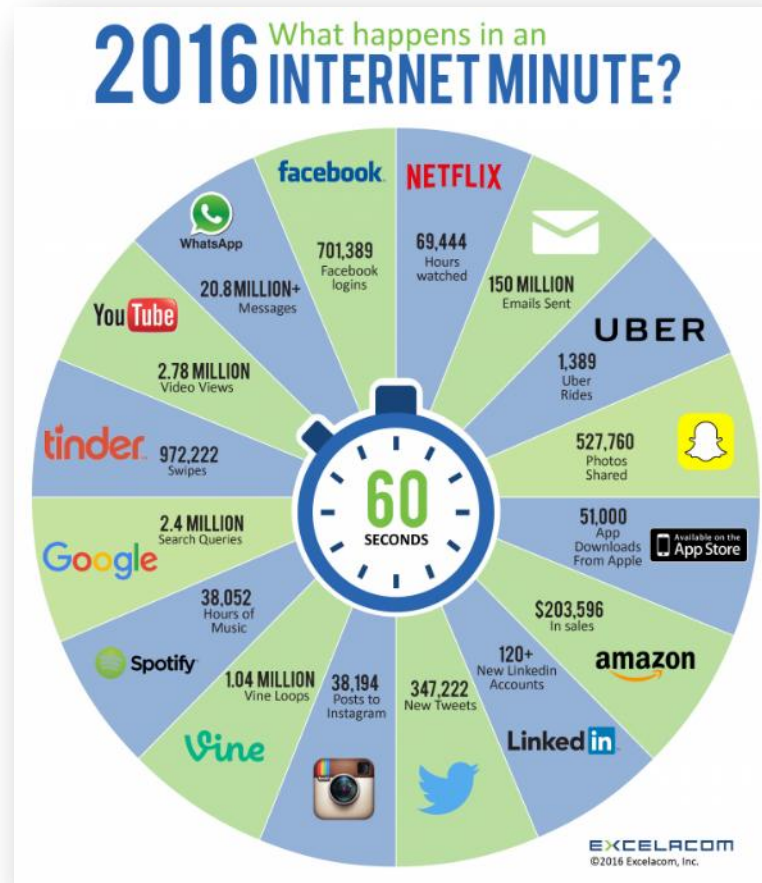
➤ Volume



Big Data

The 5Vs

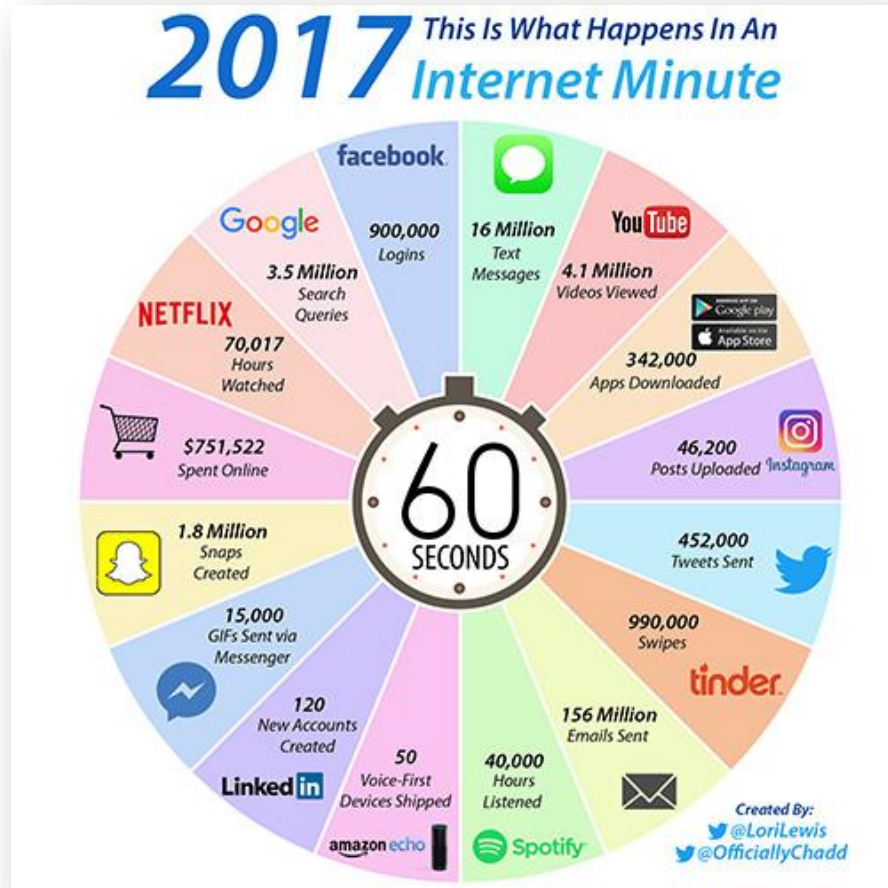
- Volume
- Velocity



Big Data

The 5Vs

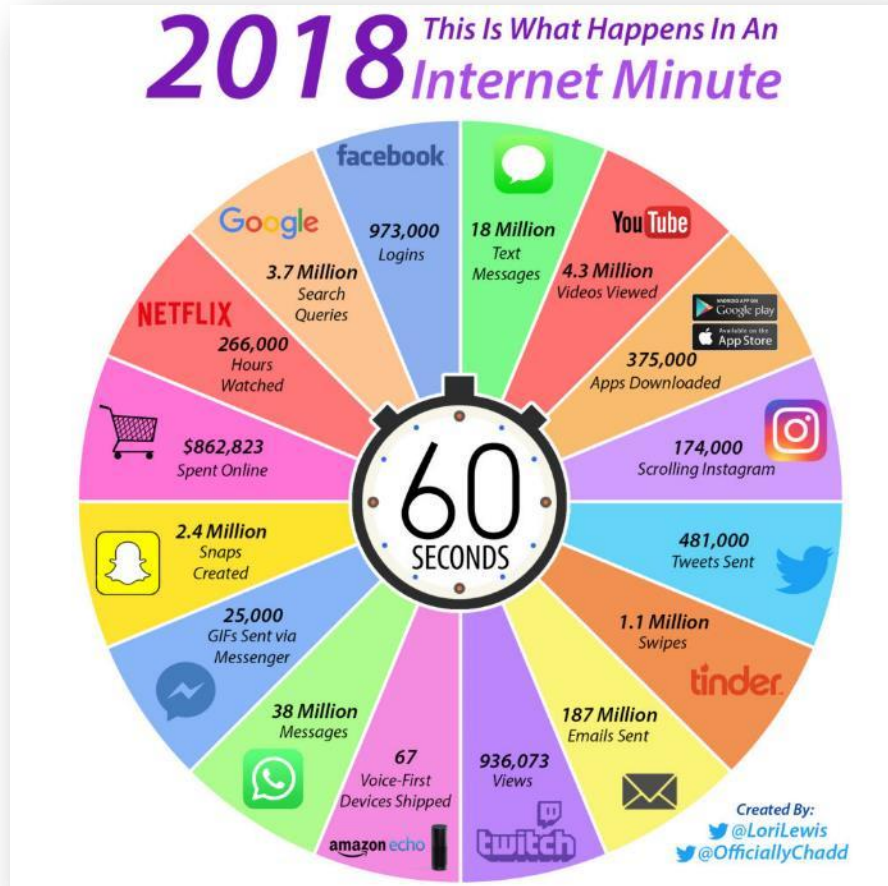
- Volume
- Velocity



Big Data

The 5Vs

- Volume
- Velocity



Big Data

The 5Vs

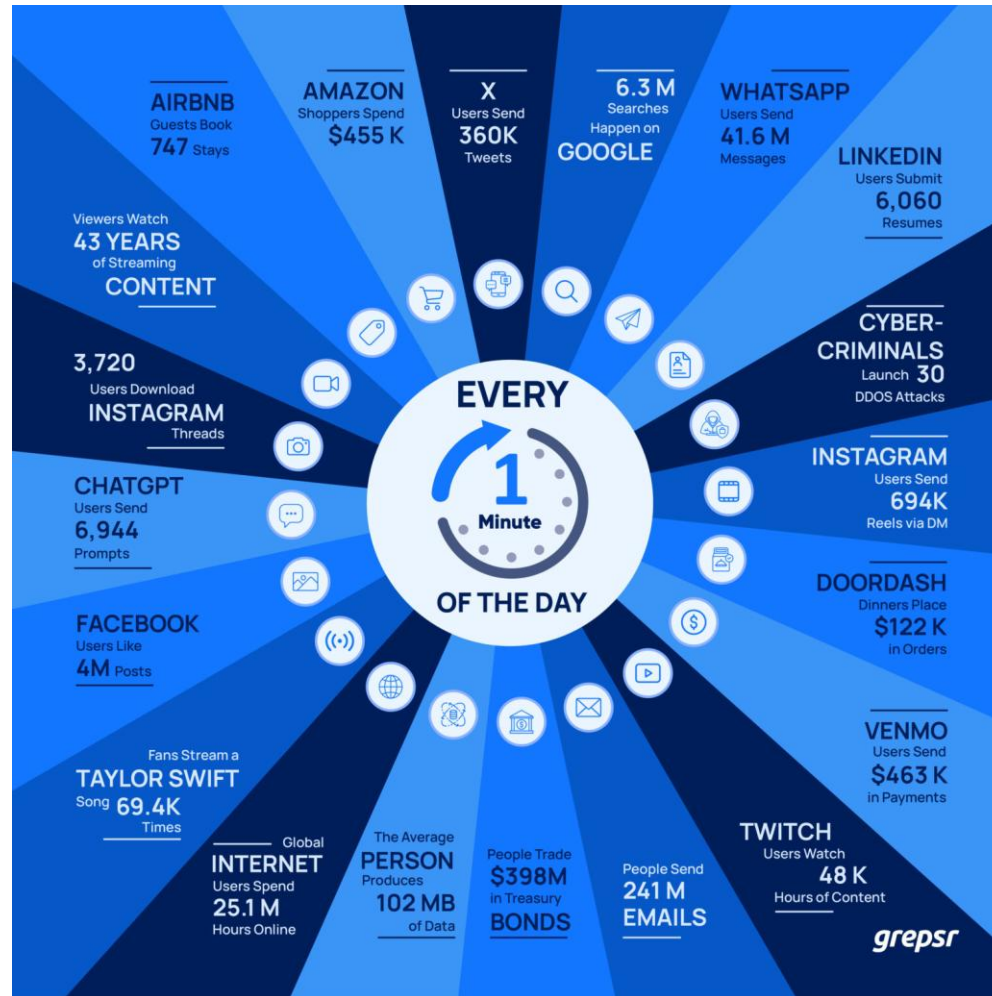
- Volume
- Velocity



Big Data

The 5Vs

- Volume
- Velocity

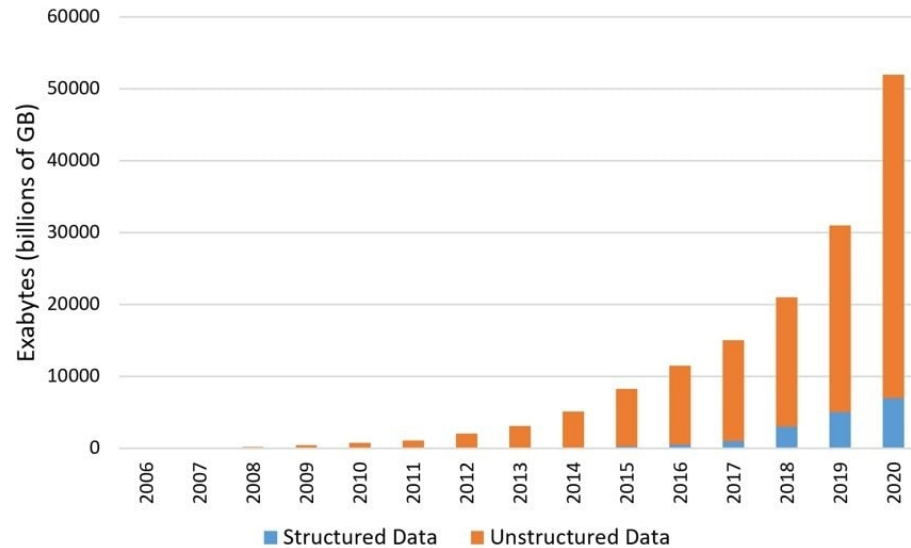


Big Data

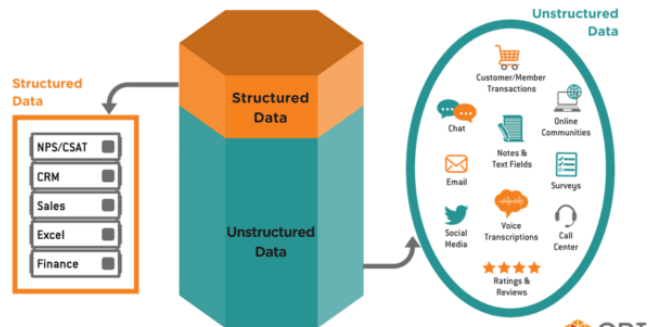
The 5Vs

- Volume
- Velocity
- Variety

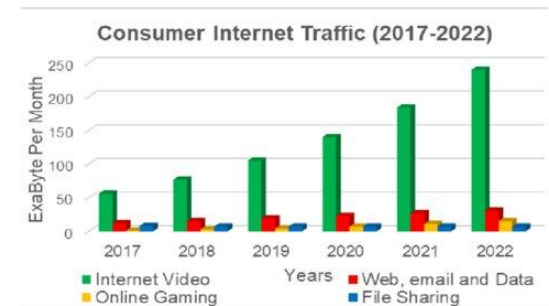
The Cambrian Explosion...of Data



What's Hiding in Your Unstructured Data?



Source: Graphic adapted from January 2018 CXPA Presentation "The Why Behind the What," Jim Kitterman



Big Data

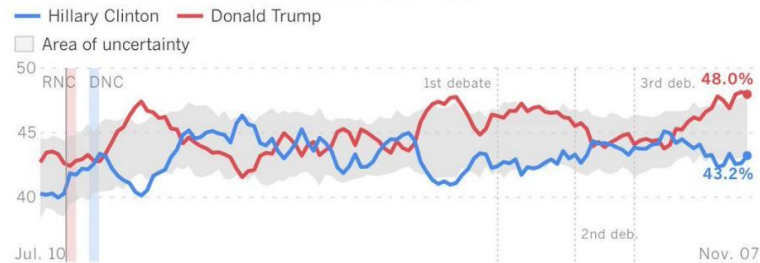
The 5Vs

- Volume
- Velocity
- Variety
- Veracity

Who's Winning? Daily track of Clinton and Trump's support

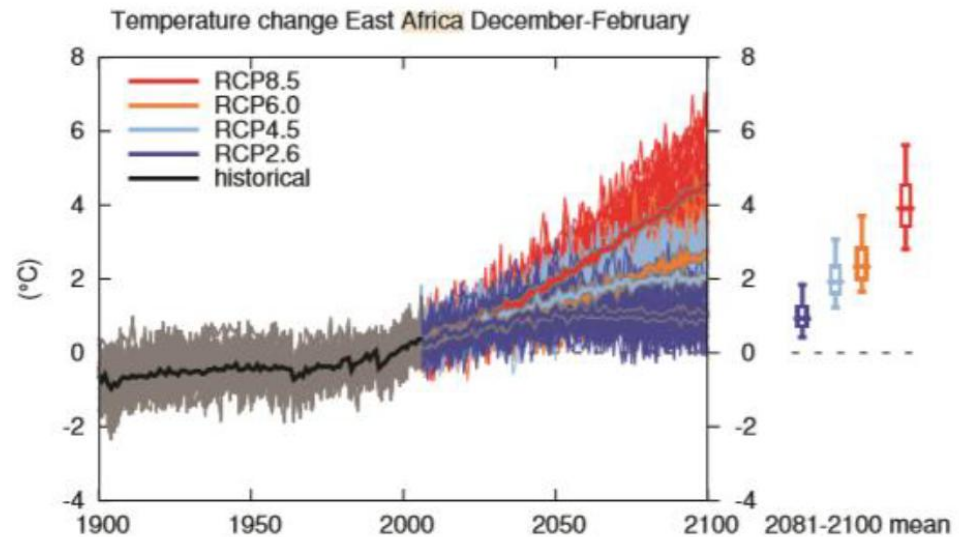
Updated daily.

[More from the poll, and why it differs from others.](#)



Note: Shaded gray area indicates the race is too close to call.

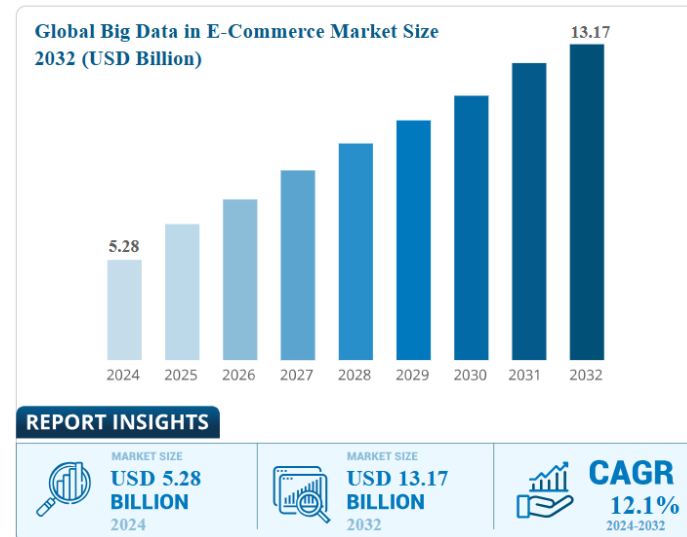
Sources: USC Dornsife/LA Times Presidential Election Daybreak Poll



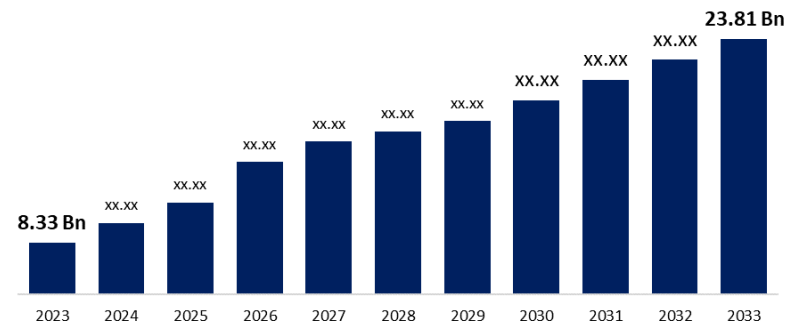
Big Data

The 5Vs

- Volume
- Velocity
- Variety
- Veracity
- Value



Global Big Data Analytics In The Energy Sector Market



Business

Big Data in Retail



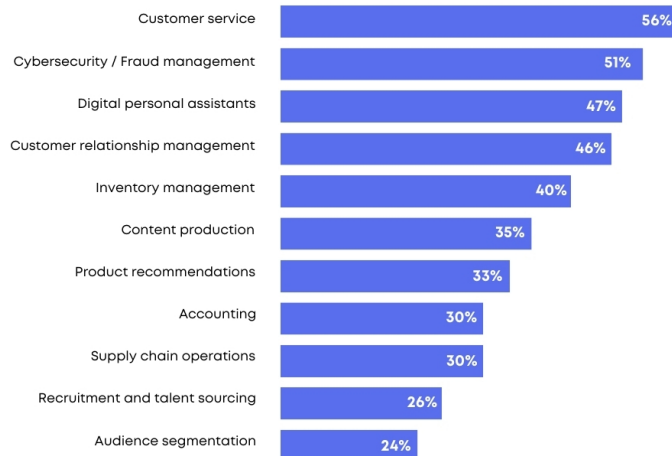
grepsr

Big Data in Finance

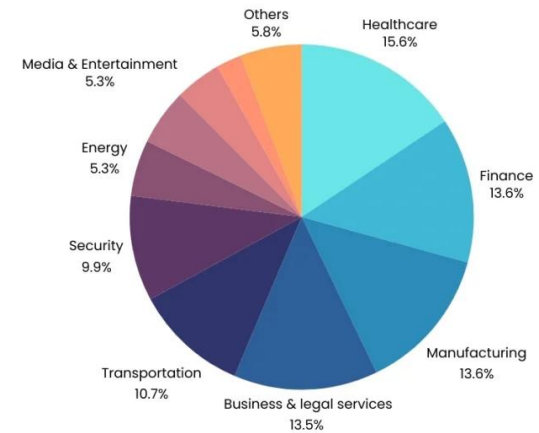


grepsr

Top Ways Business Owners Use Artificial Intelligence



AI Technologies Market Share

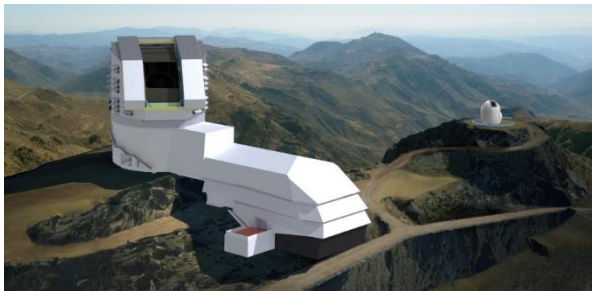


Spring 2026

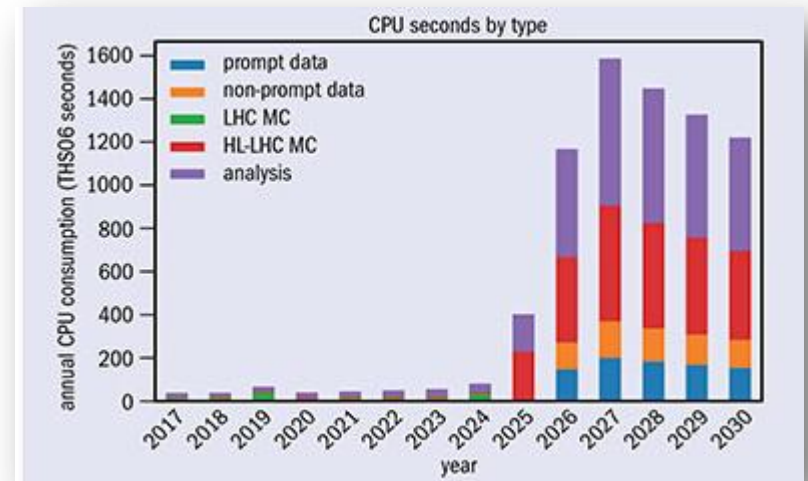
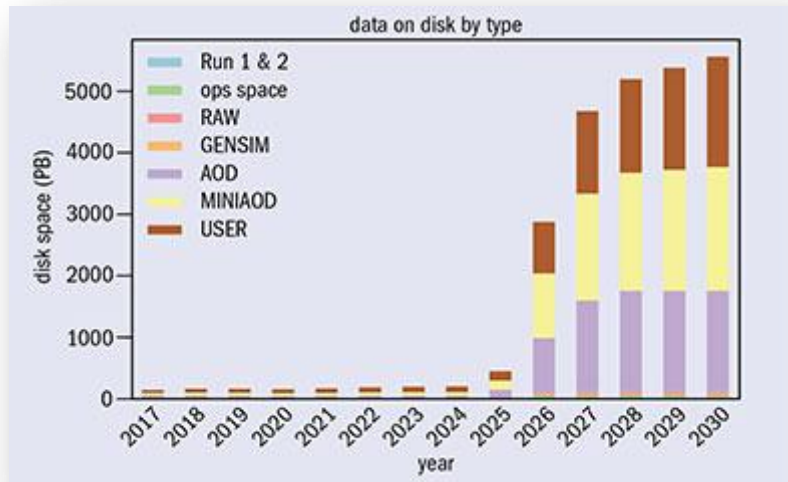
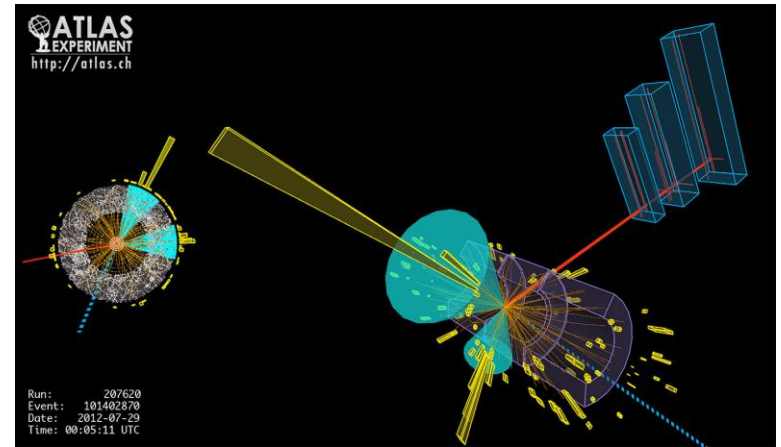
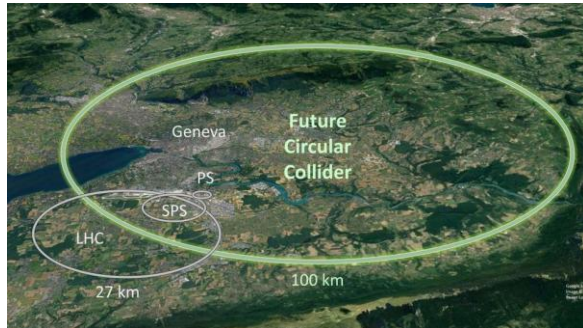


Astrophysics

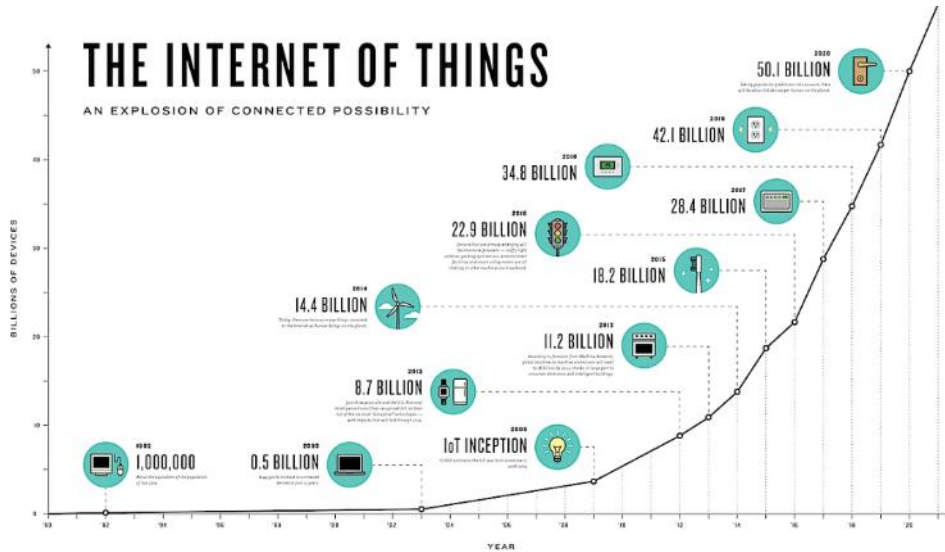
Sky Survey Project	Volume	Velocity	Variety
The Palomar Digital Sky Survey	3 PB		
Sloan Digital Sky Survey (SDSS)	50 TB	200 GB per day	Images, redshifts
Large Synoptic Survey Telescope (LSST)	~ 200 PB	10 TB per day	Images, catalogs
Square Kilometer Array (SKA)	~ 4.6 EB	150 TB per day	Images, redshifts



Nuclear physics

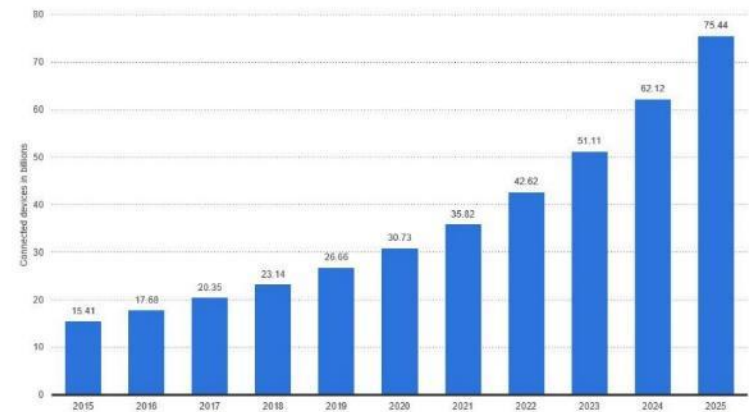


Internet-of-Things

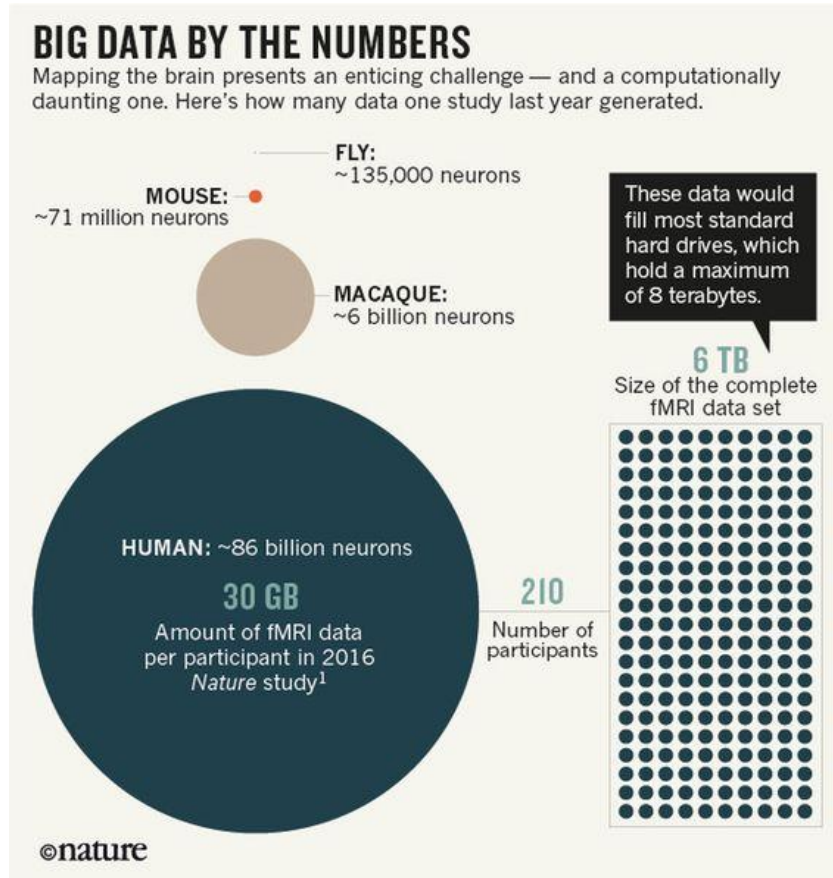


Internet of Things - number of connected devices worldwide 2015-2025

Internet of Things (IoT) connected devices installed base worldwide from 2015 to 2025 (in billions)



Biology & Medicine signals



What Is Machine Learning?

...creating and using models that learn from data...



- **data:** anything you can *measure* or *record*



- **model:** specification of a (mathematical) *relationship* between different variables

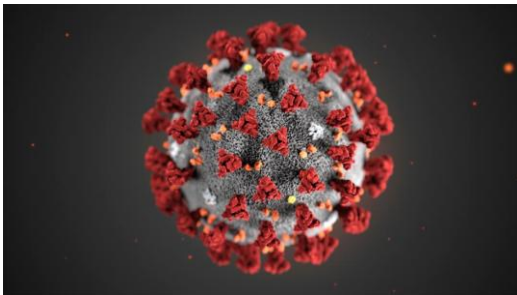


- **evaluation:** how well does the model *work?*

The evolution of AI

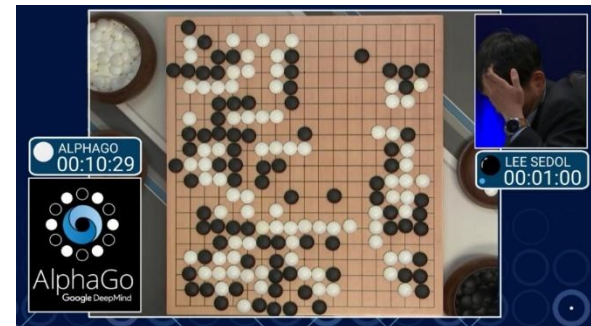
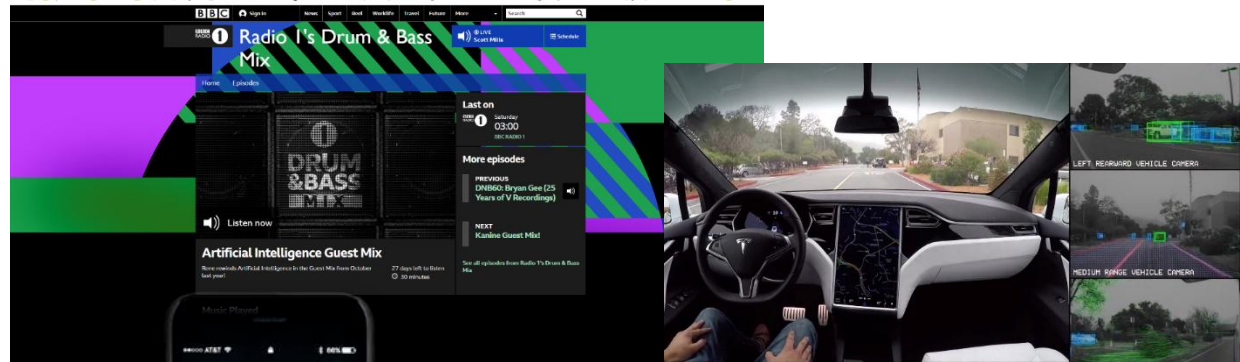


Artificial intelligence-created medicine to be used on humans for first time, BBC news, 30/1/2020



Coronavirus: Can AI Make A Difference?
Forbes, 2/2/2020

<https://www.bbc.co.uk/sounds/play/m000dsc4>



Post 2023



D Make a song about a programmer and someone who isn't a programmer.



Here is a sample song about a programmer and someone who isn't a programmer:

Verse 1:

Programmer, programmer, sitting at the screen
Typing code all day, living in a dream
Creating apps and websites, making them just right
A master of technology, always up to date

Chorus:

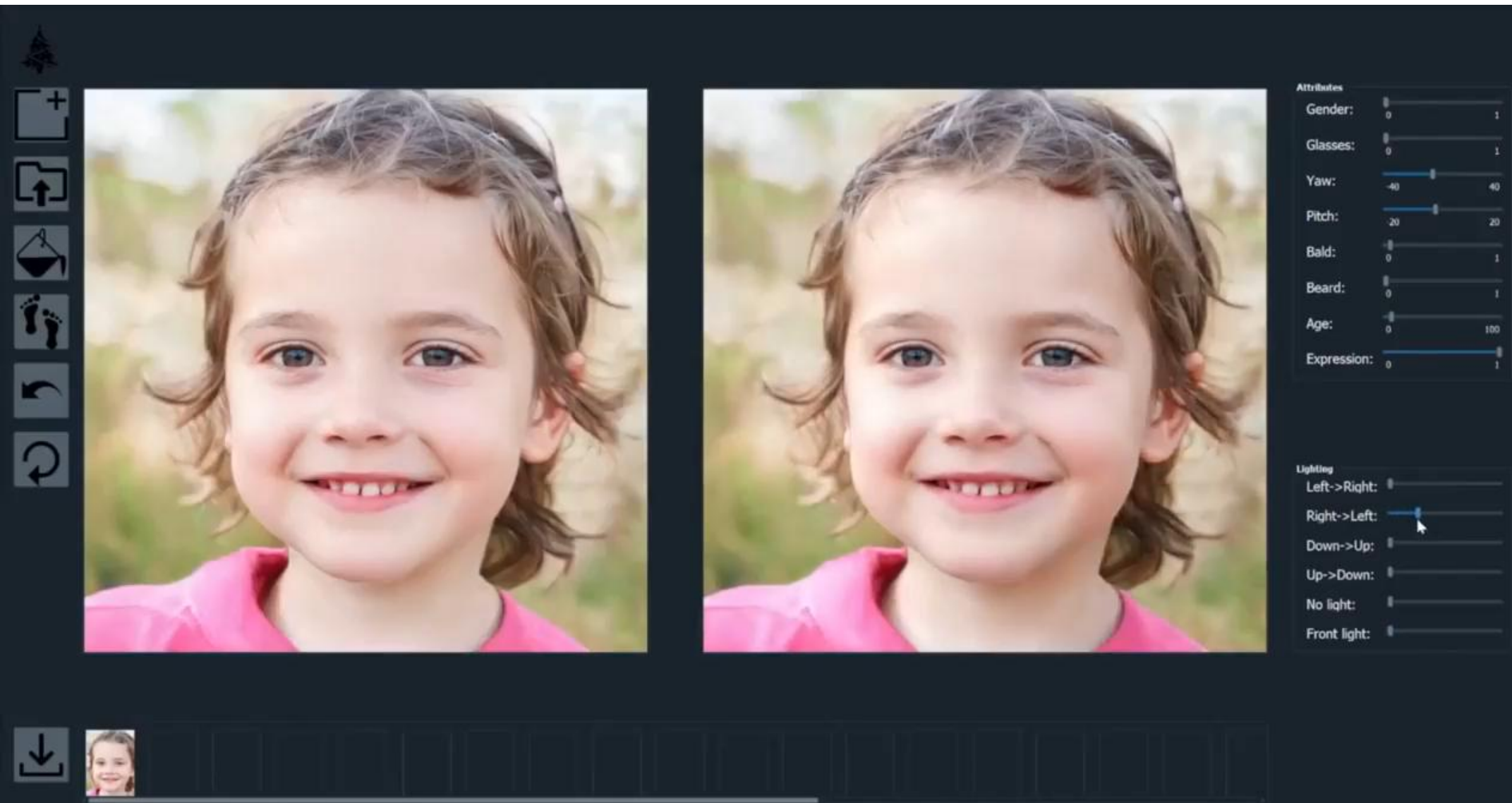
Programmer, programmer, so smart and so bright
Always coding away, day and night

Verse 2:

Non-programmer, non-programmer, not a clue
About the world of code, what it can I



“An astronaut riding a horse in a photorealistic style”



Source: [Abdal et al. 2020]



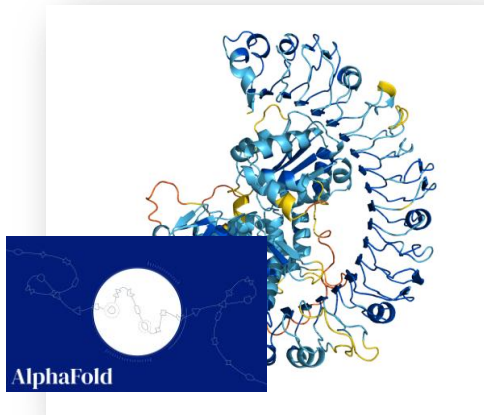
<https://youtu.be/7akzhpx0EIU?si=SwPa0Cc1mUENy69C>



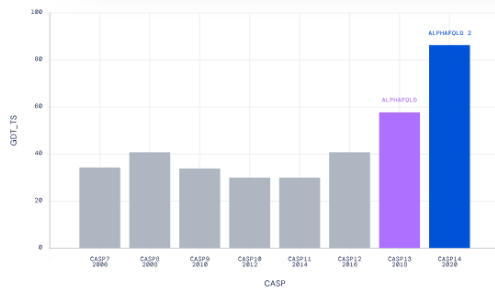
Sora by OpenAI



ML in scientific discovery



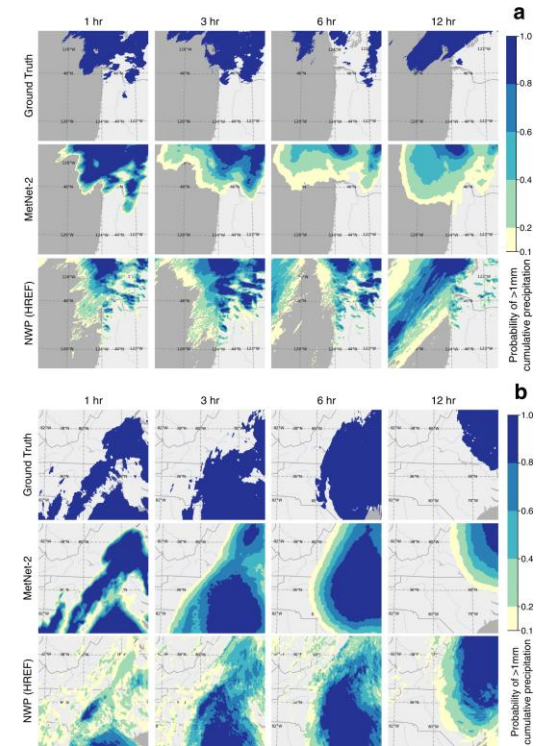
Median Free-Modelling Accuracy



Jumper, John, et al. "Highly accurate protein structure prediction with AlphaFold." *Nature*. 2021.



Valizadegan, Hamed, et al. "ExoMiner: A highly accurate and explainable deep learning classifier that validates 301 new exoplanets." *The Astrophysical Journal*, (2022)



Espeholt, Lasse, et al. "Deep learning for twelve hour precipitation forecasts." *Nature communications*, (2022).



ML in scientific discovery

THE NOBEL PRIZE IN PHYSICS 2024

Illustration: Niklas Elmehed



John J. Hopfield

Geoffrey E. Hinton

"for foundational discoveries and inventions
that enable machine learning
with artificial neural networks"

THE ROYAL SWEDISH ACADEMY OF SCIENCES

THE NOBEL PRIZE IN CHEMISTRY 2024

Illustration: Niklas Elmehed



**David
Baker**

**Demis
Hassabis**

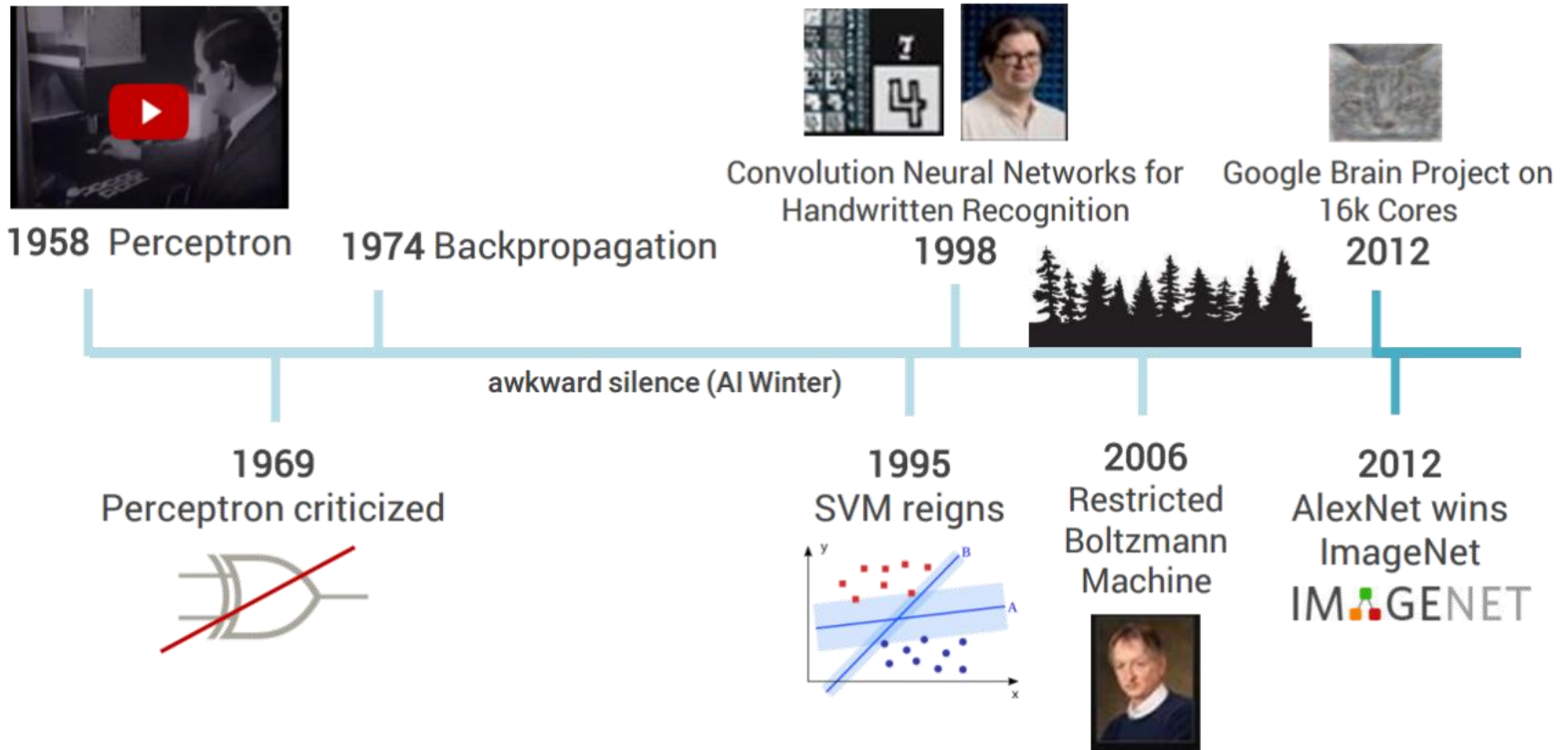
**John M.
Jumper**

"for computational
protein design"

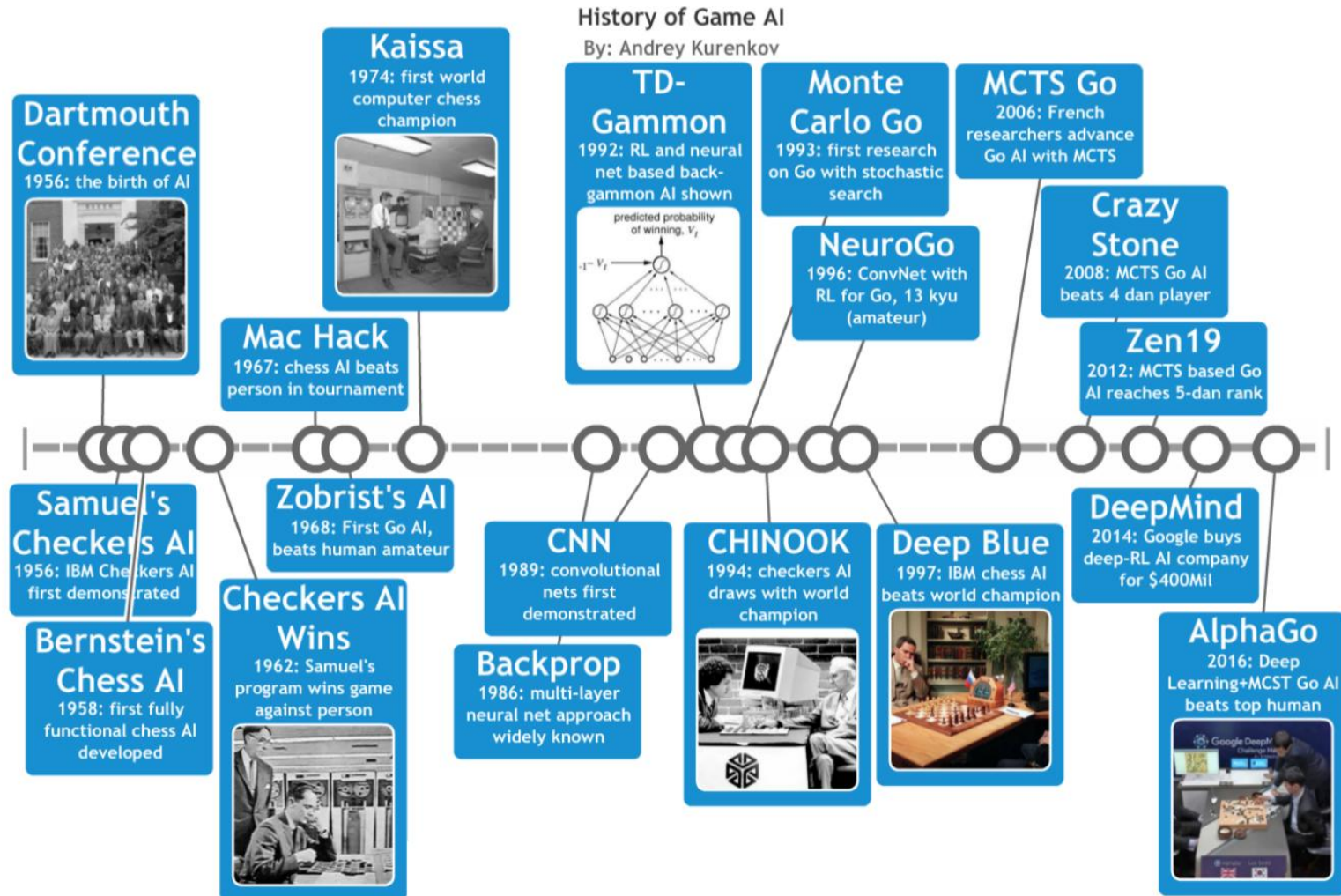
"for protein structure prediction"

THE ROYAL SWEDISH ACADEMY OF SCIENCES

Brief history of Machine Learning

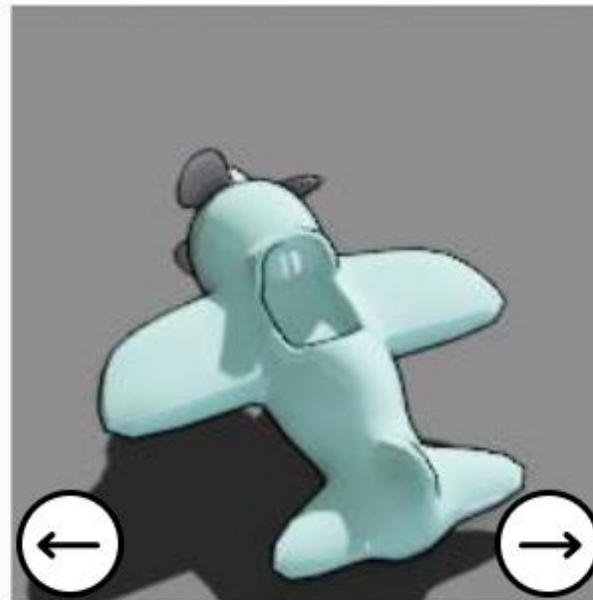


Applications in games



Verify your account

Use the arrows to rotate the object to face in the direction of the hand. (1 of 1)



Submit

4451822c1b14e92f4.4001126505



Audio



Restart



What is data science



What is data science

- Harvard Business Review: Data Scientist Is The ‘Sexiest Job Of The 21st Century’, 2012
- “Data scientist is someone who knows more statistics than a computer scientist and more computer science than a statistician.”

“There were 5 exabytes of information created between the dawn of civilization through 2003, but that much information is now created every two days.”

Eric Schmidt
Executive Chairman at Google



“Information is the oil of the 21st century, and analytics is the combustion engine.”

Peter Sondergaard
Senior Vice President and Global Head
of Research at Gartner, Inc.



“It is a capital mistake to theorize before one has data.”

Sherlock Holmes



“Data is like garbage. You’d better know what you are going to do with it before you collect it.”

Mark Twain



Customer Churn Prediction for a Telecom Company

- Objective: To develop a predictive model that identifies customers at high risk of churning.
- Data Gathering and Preparation:
 - Collect Data: Gather historical data e.g. customer demographics, call records, billing history, customer service interactions, etc.
 - Clean Data: Address missing values, remove duplicates, and correct inconsistencies.
 - Feature Engineering: Create new features e.g. average call duration, frequency of calls to customer service
- Exploratory Data Analysis (EDA):
 - Statistical Analysis: Perform statistical tests to understand the distribution of key variables, outliers.
 - Data Visualization: Use graphs and plots to uncover patterns and relationships in the data.



Customer Churn Prediction for a Telecom Company

- **Model Building and Validation:**

- Choose Algorithms: Select appropriate machine learning algorithms for analysis.
- Train Models: Use a training dataset to build various predictive models.
- Cross-Validation: Implement techniques like k-fold cross-validation to evaluate model performance.

- **Deployment and Monitoring:**

- Deploy the Model: Integrate the best-performing model into the company's IT infrastructure so it can score customers in real-time or on a scheduled basis.
- Monitor Performance: Regularly assess the model's performance, and update it as necessary to maintain accuracy over time.

- **Insights and Business Strategy:**

- Interpret Results: Identify key factors contributing to customer churn.
- Strategic Recommendations: Provide actionable insights e.g. targeted offers for at-risk customers



History

History of data science

Corporate,
Industry &
Business
People



1955s

Taylorism was adopted majorly across industrial revolution



1965s

Peter Drucker introduced the idea - knowledge worker



1975s

Kaizen and Toyota Production System is being adopted in manufacturing



1985s

Excel became the go to tool among knowledge workers

Academia,
Engg,
Math,
Computer
Hobbyists



1955s

Computers and AI were extensively researched after World War 2



1965s

The HP 9100A, the world's first desktop computer



1975s

John Tukey through his book 'Exploratory Data Analysis' showed the importance of visualizing data



1985s

Operating Systems made its debut in the market with MS Windows

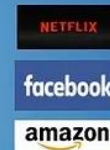
1995s

Internet makes the world connected



2000s

DOTCom Boom - every company take their business online



2005s

Service industry transformed customer experience and they improved with data



2010s

Smartphones intensified customer obsession and era of on-demand economy starts



2015s

Machine Learning and AI becomes mainstream. Computation intensified with GPUs





Introduction to Python

Overview of Python

Disclaimer!!!

- I am not a programmer
- Most of you are better than me (in python 😊)
- *Programming is the way to translate algorithms into software*



Key Technologies



Python



**Jupyter
Notebooks**



**Google
Colab**

Basic syntax

“Hello, World”

- C

```
#include <stdio.h>

int main(int argc, char ** argv)
{
    printf("Hello, World!\n");
}
```

- Java

```
public class Hello
{
    public static void main(String argv[])
    {
        System.out.println("Hello, World!");
    }
}
```

- now in Python

```
print "Hello, World!"
```



Formatting

```
# The pound sign marks the start of a comment. Python itself  
# ignores the comments, but they're helpful for anyone reading the code.  
for i in [1, 2, 3, 4, 5]:  
    print(i)                # first line in "for i" block  
    for j in [1, 2, 3, 4, 5]:  
        print(j)           # first line in "for j" block  
        print(i + j)       # last line in "for j" block  
    print(i)               # last line in "for i" block  
print("done looping")
```

- Whitespace is ignored inside parentheses and brackets

```
long_winded_computation = (1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10 + 11 + 12 +  
                           13 + 14 + 15 + 16 + 17 + 18 + 19 + 20)
```

```
list_of_lists = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

```
easier_to_read_list_of_lists = [[1, 2, 3],  
                                [4, 5, 6],  
                                [7, 8, 9]]
```



Modules

- Certain features of Python are not loaded by default. These include both features that are included as part of the language as well as third-party features that you download yourself

```
import re  
my_regex = re.compile("[0-9]+", re.I)
```

- If you already had a different re in your code, you could use an alias:

```
import re as regex  
my_regex = regex.compile("[0-9]+", regex.I)
```



Modules

- Also useful for not writing a lot

```
import matplotlib.pyplot as plt

plt.plot(...)
```

- Import components explicitly

```
from collections import defaultdict, Counter
lookup = defaultdict(int)
my_counter = Counter()
```

- Don't be a bad person

```
match = 10
from re import *      # uh oh, re has a match function
print(match)          # "<function match at 0x10281e6a8>"
```



Functions

- Python functions are *first-class*, which means that we can assign them to variables and pass them into functions just like any other arguments:

```
def apply_to_one(f):  
    """Calls the function f with 1 as its argument"""  
    return f(1)
```

```
my_double = double           # refers to the previously defined function  
x = apply_to_one(my_double)  # equals 2
```

- It is also easy to create short anonymous functions, or *lambdas*:

```
y = apply_to_one(lambda x: x + 4)    # equals 5
```



Variable types

- Integer (`int`)
- Float (`float`)
- String (`str`)
- Boolean (`bool`)
- Complex (`complex`)
- [...]
- User defined (`classes`)

- A variable is assigned using the `=` operator;
i.e:

```
In: intVar = 5
     floatVar = 3.2
     stringVar = "Food"
     print(intVar)
     print(floatVar)
     print(stringVar)
```

```
Out: In [4]: runfi
      1b690/.spyder
      5
      3.2
      Food
```

- The `print()` function is used to print something to the screen.

- You can always check the type of a variable using the `type()` function.

```
In: variable = 100
     print(type(variable))
```

```
Out: <class 'int'>
```



Type Casting

- Variables can be *cast* to a different type.

```
In: share_of_rent = 295.50/2.0
     print("1:", share_of_rent)
     print(type(share_of_rent))
     rounded_share = int(share_of_rent)
     print("2:", rounded_share)
     print(type(rounded_share))
```

```
Out: 1: 147.75
      <class 'float'>
      2: 147
      <class 'int'>
```



Arithmetic operators

The arithmetic operators:

- Addition: $+$
- Subtract: $-$
- Multiplication: $*$
- Division: $/$
- Power: $**$



Comparison operators

- Greater than: >
- Lesser than: <
- Greater than or equal to: >=
- Lesser than or equal to: <=
- Is equal to: ==

- Write a couple of operations using comparison operators; i.e.

```
In: intVar = 5
     floatVar = 3.2
     stringVar = "Food"

     if intVar > floatVar:
         print("Yes")

     if intVar == 5:
         print("A match!")
```

```
Out: In [9]: run
      1b690/.spyd
      Yes
      A match!
```



For loops

- The for loop is used to iterate over elements in a sequence, and is often used when you have a piece of code that you want to repeat a number of times.
- For loops essentially say:

“For all elements in a sequence, do something”



An example

- We have a list of species:

```
species = ['dog', 'cat', 'shark', 'falcon', 'deer', 'tyrannosaurus rex']  
for i in species:  
    print(i)
```

- The command underneath the list then cycles through each entry in the species list and prints the animal's name to the screen. Note: The `i` is quite arbitrary. You could just as easily replace it with `'animal'`, `'t'`, or anything else.

```
In [1]: runfile('//is  
dog  
cat  
shark  
falcon  
deer  
tyrannosaurus rex
```



Another example

- We can also use for loops for operations other than printing to a screen. For example:

```
numbers = [1, 20, 18, 5, 15, 160]
total = 0
for value in numbers:
    total = total + value
print(total)
```

```
In [4]: runfile('
219
```



The range() function

- The `range()` function generates a list of numbers, which is generally used to iterate over within for loops.
- The `range()` function has two sets of parameters to follow:

`range(stop)`

stop: Number of integers
(whole numbers) to generate,
starting from zero. i.e:

```
for i in range(5):  
    print(i)
```

```
In [6]: runfile('/  
0  
1  
2  
3  
4
```

`range([start], stop[, step])`

start: Starting number of the sequence.

stop: Generate numbers up to, but not including this number.

step: Difference between each number in the sequence

i.e.:

```
for i in range(3,6):  
    print(i)
```

```
In [7]: runfile('/  
3  
4  
5
```

```
for i in range(4, 10, 2):  
    print(i)
```

```
In [8]: runfile  
4  
6  
8
```

Note:

- All parameters must be integers.
- Parameters can be positive or negative.
- The `range()` function (and Python in general) is 0-index based, meaning list indexes start at 0, not 1. eg. The syntax to access the first element of a list is `mylist[0]`. Therefore the last integer generated by `range()` is up to, but not including, stop.



Example

- Create an empty list.

```
new_list = []
```

- Use the range() and append() functions to add the integers 1-20 to the empty list.

```
for i in range(1, 21):  
    new_list.append(i)
```

- Print the list to the screen, what do you have?

```
print(new_list)
```

Output [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]
:

Loop control

- The `break()` function
- To terminate a loop, you can use the `break()` function.
- The `break()` statement breaks out of the innermost enclosing `for` or `while` loop.

```
for i in range(1, 10):  
    if i == 3:  
        break  
    print(i)
```

1
2

- The `continue()` statement is used to tell Python to skip the rest of the statements in the current loop block, and then to continue to the next iteration of the loop.

```
for i in range(1, 10):  
    if i == 3:  
        continue  
    print(i)
```

1
2
4
5
6
7
8
9



While loops

- The while loop tells the computer to do something as long as a specific condition is met.
- When working with while loops, its important to remember the nature of various operators.
- While loops use the `break()` and `continue()` functions in the same way as a for loop does.

```
species = ['dog', 'cat', 'shark', 'falcon', 'deer', 'tyrannosaurus rex']  
i = 0  
while i < 3:  
    print(species[i])  
    i = i + 1
```

```
dog  
cat  
shark
```



Nested loops

- In some situations, you may want a loop within a loop; this is known as a nested loop.

```
In:  for x in range(1, 11):  
      for y in range(1, 11):  
          print('%d * %d = %d' %(x, y, x*y))
```

Out:

```
1 * 1 = 1  
1 * 2 = 2  
1 * 3 = 3  
1 * 4 = 4  
1 * 5 = 5  
1 * 6 = 6  
1 * 7 = 7
```



Conditionals

- There are three main conditional statements in Python; `if`, `else`, `elif`.
- We have already used `if` when looking at `while` loops.

```
In: school_night = True
    if school_night == True:
        print("No beer")
    else:
        print("You may have beer")
```

Out: No beer

```
In: school_night = False
    if school_night == True:
        print("No beer")
    else:
        print("You may have beer")
```

Out: You may have beer



An example of elif

```
In: Lew_is_tired = False
    Lew_is_hungry = True
    if Lew_is_tired is True:
        print("Lew has to teach")
    elif Lew_is_hungry is True:
        print("No food for Lew")
    else:
        print("Go on, have a biscuit")
```

```
Out: No food for Lew
```



Reading and writing to files in Python

- Before you can work with a file, you first have to open it using Python's in-built `open()` function.
- The `open()` function takes two arguments; the name of the file that you wish to use and the mode for which we would like to open the file

```
practiceFile = open('Practice_file_for_IOC.txt', 'r')
```

- By default, the `open()` function opens a file in 'read mode'; this is what the 'r' above signifies.
- There are a number of different file opening modes. The most common are: 'r'=read, 'w'=write, 'r+'=both reading and writing, 'a'=appending.
- Likewise, once you're done working with a file, you can close it with the `close()` function.

```
practiceFile.close()
```



The read() function

- However, you don't need to use any loops to access file contents. Python has three in-built file reading commands:

1. `<file>.read()` = Returns the entire contents of the file as a single string:

```
practiceFile = open('practice_file.txt', 'r')
print(practiceFile.read())
```

```
In [44]: runfile('///isad.isadroot.e
User/Desktop')
The first line of text
The second line of text
The third line of text
The fourth line of text
I'm bored now, you get the idea
```

2. `<file>.readline()` = Returns one line at a time:

```
practiceFile = open('practice_file.txt', 'r')
print(practiceFile.readline())
```

```
In [51]: runfile('///isad.i
User/Desktop')
The first line of text
```

3. `<file>.readlines()` = Returns a list of lines:

```
practiceFile = open('practice_file.txt', 'r')
print(practiceFile.readlines())
```

```
In [52]: runfile('///isad.isadroot.ex.ac.uk/UOE/User/Desktop/IOC_test.py',
wdir='///isad.isadroot.ex.ac.uk/UOE/User/Desktop')
['The first line of text\n', 'The second line of text\n', 'The third line of
text\n', 'The fourth line of text\n', "I'm bored now, you get the idea\n"]
```

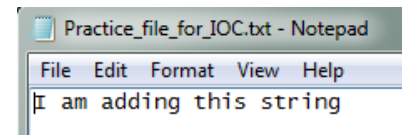


The write() function

- Likewise, there are two similar in-built functions for getting Python to write to a file:

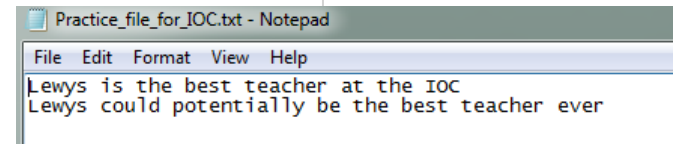
1. `<file>.write()` = Writes a specified sequence of characters to a file:

```
practiceFile = open('Practice_file_for_IOC.txt', 'w')
practiceFile.write('I am adding this string')
```



2. `<file>.writelines()` = Writes a list of strings to a file:

```
testList = ['Lewys is the best teacher at the IOC\n', 'Lewys could potentially be the best teacher ever\n']
practiceFile = open('Practice_file_for_IOC.txt', 'w')
practiceFile.writelines(testList)
```



- Important: Using the `write()` or `writelines()` function will overwrite anything contained within a file, if a file of the same name already exists in the working directory.



The append() function

- If you do not want to overwrite a file's contents, you can use the `append()` function.
- To append to an existing file, simply put `'a'` instead of `'r'` or `'w'` in the `open()` when opening a file.

```
practiceFile = open('Practice_file_for_IOC.txt', 'a')  
testLine = '\nI told you I was the best'  
practiceFile.write(testLine)
```

